SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	VIVETHA B	**Reg. No.:**	192424170
Date:		**Duration:**	60 minutes

Code of Conduct

I VIVETHA B 192424170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VIVETHA B

Annexure A – Architecture Design Assignment

PHISHING EMAIL DETECTION AND CYBER THREAT INTELLIGENCE SYSTEM

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.

The main objective of the **Phishing Email Detection and Cyber Threat Intelligence System** is to develop an intelligent and reliable system that can automatically identify phishing emails and detect potential cyber threats before they cause harm to users or organizations. The system analyzes different characteristics of an email, including the sender's information, subject, email content, suspicious keywords, embedded URLs, attachments, and other technical indicators to determine whether the email is legitimate, suspicious, or malicious.

It uses **machine learning and cyber threat intelligence techniques** to identify known and emerging phishing patterns and improve the accuracy of threat detection. The system can also analyze suspicious URLs, domains, IP addresses, and other indicators of compromise by comparing them with available threat intelligence information. When a potential phishing email or cyber threat is detected, the system

provides an appropriate classification, risk level, confidence score, and alert to help users understand the threat.

The system aims to reduce the risk of credential theft, malware infection, financial fraud, identity theft, and unauthorized access to sensitive information. It also helps security teams monitor suspicious activities, investigate detected threats, and take appropriate preventive actions. Overall, the system is designed to provide **early threat detection, improved email security, automated analysis, real-time alerts, and better cybersecurity awareness**, thereby creating a safer communication environment for individuals and organizations.

- Analyze the functional and non-functional requirements that influence the software architecture.

  Functional Requirements:

  - User login and authentication.
  - Upload or enter email content for analysis.
  - Analyze sender details and email headers.
  - Extract suspicious keywords and email features.
  - Analyze URLs, domains, and attachments.
  - Detect phishing emails using machine learning.
  - Classify emails as **Phishing, Suspicious, or Legitimate**.
  - Generate a threat/risk score.
  - Integrate Cyber Threat Intelligence (CTI) sources.
  - Detect malicious IP addresses, URLs, and domains.
  - Generate alerts for detected threats.
  - Store and display detection history.
  - Generate security reports and dashboards.

  Non-Functional Requirements:

  - **Security:** Protect email data and user information.
  - **Accuracy:** Provide reliable phishing detection results.
  - **Performance:** Analyze emails and provide results quickly.
  - **Scalability:** Handle increasing users and email volumes.
  - **Reliability:** Operate consistently with minimal failures.

- **Availability:** Keep the system accessible when required.
- **Usability:** Provide a simple and user-friendly interface.
- **Maintainability:** Make the system easy to update and modify.
- **Privacy:** Protect confidential email and user information.
- **Extensibility:** Allow new threat intelligence sources and ML models to be added.
- **Fault Tolerance:** Continue operating even when some components fail.

- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

- **Scalability:** The system should handle an increasing number of users, emails, and cyber threat intelligence data without significant performance loss.
- **Maintainability:** The system should be easy to modify, update, debug, and maintain when new phishing techniques or detection methods are introduced.
- **Reliability:** The system should consistently provide correct phishing detection results and operate with minimal failures.
- **Availability:** The system should remain accessible and operational whenever users need to analyze suspicious emails.
- **Performance:** The system should analyze emails quickly and provide detection results and threat alerts with minimal delay.
- **Security:** The system should protect sensitive email content, user information, credentials, and threat intelligence data from unauthorized access.
- **Accuracy:** The system should correctly identify phishing and legitimate emails while minimizing false positives and false negatives.
- **Usability:** The system should provide a simple and user-friendly interface for submitting emails and understanding detection results.
- **Privacy:** The system should ensure that confidential email data is securely handled and not unnecessarily exposed.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture)

- **Layered Architecture** is suitable for separating the system into different levels such as presentation, application, detection, threat intelligence, and database layers.
- **Client-Server Architecture** allows users to access the phishing detection system through a web browser, while the server performs email analysis and threat detection.
- The **Presentation Layer** provides the user interface and dashboard.
- The **Application Layer** manages user requests and system operations.
- The **Phishing Detection Layer** analyzes email content and uses machine learning models to classify emails.
- The **Cyber Threat Intelligence Layer** checks suspicious URLs, domains, IP addresses, and other threat indicators.
- The **Database Layer** stores user information, email analysis results, threat intelligence data, and detection history.
- This architecture provides **scalability, security, maintainability, reliability, and better performance**.
- Therefore, a **Hybrid Layered Client-Server Architecture** is appropriate for the proposed **Phishing Email Detection and Cyber Threat Intelligence System**.

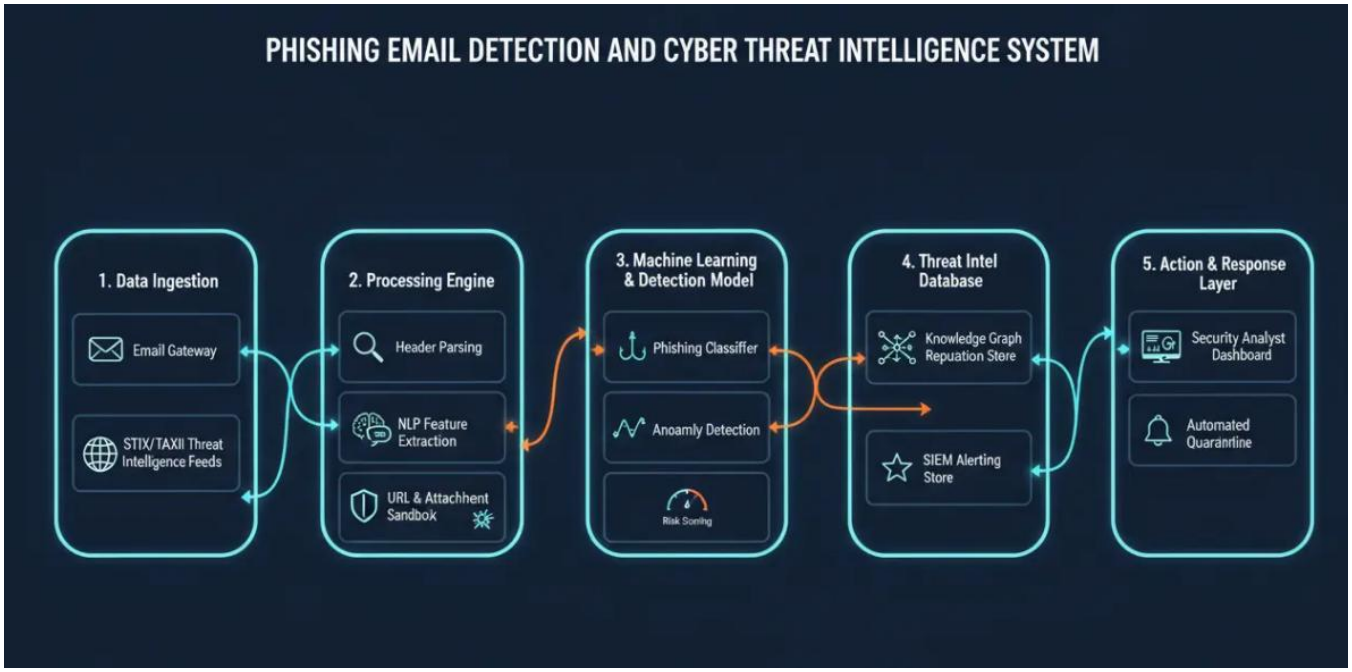
- Justify why the selected architecture is the most suitable for the given problem.

- **Easy to understand:** Divides the system into separate layers, making the architecture simple and organized.
- **Modular design:** Each component, such as phishing detection and threat intelligence, can be developed separately.
- **Better security:** Sensitive email data and user information can be protected through authentication and access control.
- **Scalability:** The system can handle increasing numbers of users and emails.
- **Maintainability:** Individual modules can be updated or modified without affecting the entire system.
- **Good performance:** Email analysis and machine learning processing can be performed on the server.
- **Centralized management:** Detection models, databases, and threat intelligence sources can be managed centrally.
- **Machine Learning support:** The architecture can easily integrate ML models for phishing email classification.

- **Threat Intelligence integration:** External threat intelligence services can be connected to identify malicious URLs, IP addresses, and domains.
- **Reliable communication:** Client-server communication allows users to submit emails and receive detection results efficiently.
- **Cost-effective:** It can be implemented using commonly available web technologies and cloud services.
- **Suitable for the problem:** Overall, the hybrid architecture provides the required **security, scalability, maintainability, reliability, and performance** for the Phishing Email Detection and Cyber Threat Intelligence System.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.



The **Phishing Email Detection and Cyber Threat Intelligence System** follows a layered architecture consisting of five major components. First, the **Data Ingestion Layer** collects incoming emails and threat intelligence data from sources such as email gateways and STIX/TAXII feeds. Next, the **Processing Engine** analyzes the collected data by performing header parsing, NLP-based feature extraction, and URL/attachment sandbox analysis.

The processed features are then passed to the **Machine Learning and Detection Model**, which uses a phishing classifier, anomaly detection, and risk scoring to identify suspicious emails and determine their threat level. The **Threat Intelligence Database** enriches the detection process by storing knowledge graphs and SIEM alert information, allowing the system to correlate detected threats with known indicators of compromise.

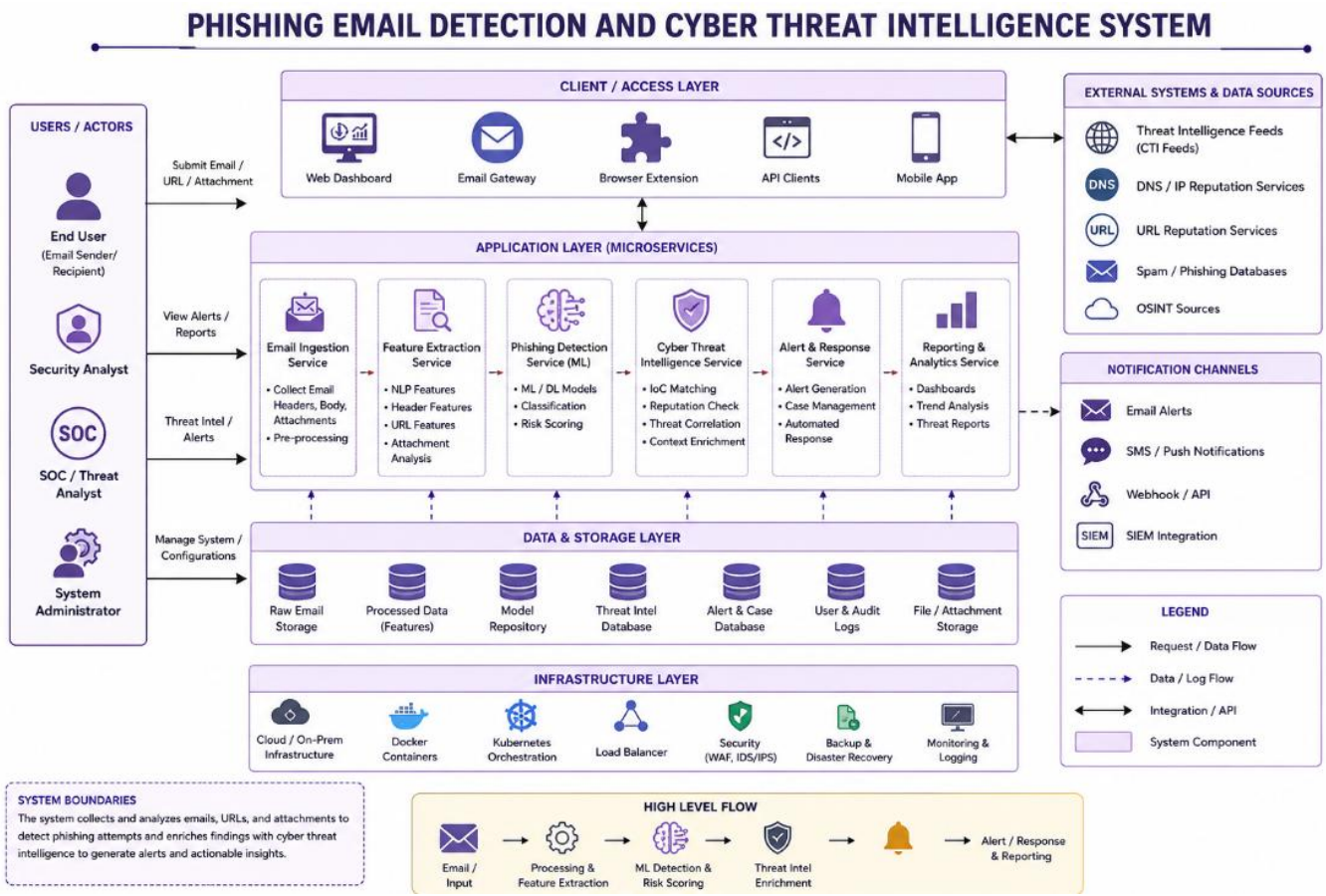
Finally, the **Action and Response Layer** provides security analyst dashboards and automated quarantine capabilities. Based on the detected risk, the system can generate alerts and take appropriate actions automatically. This architecture enables **real-time phishing detection, threat intelligence enrichment, automated response, and continuous security monitoring**, making the system scalable and effective for cyber threat detection.

- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.

- **Presentation Layer:** Web interface, login page, email upload page, threat detection dashboard, and result display.
- **Authentication Layer:** User login, registration, authentication, authorization, and role management.
- **Application/Business Logic Layer:** Handles user requests, email processing, detection workflow, and system operations.
- **Email Analysis Layer:** Extracts email content, sender details, headers, keywords, URLs, and attachments.
- **Feature Extraction Module:** Identifies important features used for phishing detection.
- **Machine Learning Detection Layer:** Uses ML models to classify emails as **Phishing, Suspicious, or Legitimate**.
- **Cyber Threat Intelligence Layer:** Checks URLs, domains, IP addresses, hashes, and other Indicators of Compromise (IoCs).
- **Threat Intelligence APIs:** Connects with external threat intelligence services to obtain updated information about malicious indicators.
- **Alert and Notification Module:** Generates alerts when phishing or other cyber threats are detected.
- **Reporting and Dashboard Module:** Displays risk scores, detection history, threat statistics, and reports.

- **Database Layer:** Stores user details, email analysis results, threat indicators, detection history, logs, and ML-related information.
- **External Systems:** Email services, threat intelligence platforms, antivirus/security services, and cloud services.
- **APIs:** REST APIs or HTTPS APIs are used to communicate between the frontend, backend, ML model, database, and external threat intelligence services.
- **Communication Mechanisms:** HTTPS for secure communication, REST/JSON for API communication, and secure database connections for data storage and retrieval.
- **Logging and Monitoring:** Records system activities, errors, detection events, and security incidents for monitoring and auditing.

- Use appropriate software architecture notations and labels.



4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component

- **User Interface (UI):** Provides an interface for users to upload emails, view analysis results, check threat levels, and monitor security alerts.
- **Authentication Module:** Verifies user identity and manages login, registration, roles, and access permissions.
- **Email Processing Module:** Receives the email and extracts important information such as sender, subject, body, URLs, headers, and attachments.
- **Feature Extraction Module:** Identifies suspicious keywords, unusual email patterns, malicious links, domain information, and other features required for detection.
- **Machine Learning Module:** Analyzes extracted features and classifies the email as **Phishing, Suspicious, or Legitimate**.
- **Cyber Threat Intelligence Module:** Checks URLs, IP addresses, domains, hashes, and other Indicators of Compromise (IoCs) against threat intelligence sources.
- **Risk Assessment Module:** Combines ML predictions and threat intelligence results to calculate the overall threat or risk score.
- **Alert and Notification Module:** Sends warnings or notifications when a suspicious or phishing email is detected.
- **Database:** Stores user details, email analysis results, threat intelligence data, detection history, alerts, and system logs.
- **API Gateway/Backend:** Acts as the communication layer between the user interface, detection modules, database, and external APIs.
- **External Threat Intelligence APIs:** Provide updated information about malicious URLs, IP addresses, domains, malware, and other cyber threats.
- **Dashboard and Reporting Module:** Presents detection results, risk scores, statistics, history, and security reports in an understandable format.
- **Logging and Monitoring Module:** Records system activities, errors, alerts, and detection events to support monitoring and troubleshooting.

Interaction Flow:

The user uploads an email → **UI** sends it to the **Backend** → **Email Processing** extracts information → **Feature Extraction** identifies relevant features → **Machine Learning Module** analyzes the email →

Cyber Threat Intelligence Module checks external threat data → **Risk Assessment** determines the threat level → **Database** stores the result → **Dashboard/Alert Module** displays the result and notifies the user.

- Explain how the components communicate and exchange data.

- The **User Interface (UI)** communicates with the **Backend Server** through **HTTPS/REST APIs**.
- The user uploads an email, and the UI sends the email data to the backend in a structured format such as **JSON**.
- The **Backend** forwards the email to the **Email Processing Module** for analysis.
- The **Email Processing Module** extracts sender details, subject, body, URLs, headers, and attachments.
- The extracted information is sent to the **Feature Extraction Module**, which converts the email into features required by the ML model.
- The **Machine Learning Module** receives these features and returns the predicted classification, such as **Phishing, Suspicious, or Legitimate**, along with a confidence score.
- The **Cyber Threat Intelligence Module** communicates with external threat intelligence services through **APIs** to check suspicious URLs, IP addresses, domains, and hashes.
- The **Risk Assessment Module** combines the ML prediction and threat intelligence results to calculate the overall threat level.
- The **Backend** stores analysis results, user information, threat data, and logs in the **Database** using secure database connections.
- The **Alert Module** receives high-risk results from the backend and sends appropriate warnings to the user.
- The **Dashboard** retrieves the final results from the backend and displays the classification, risk score, detected threats, and analysis history.
- All communication between components should use **secure protocols such as HTTPS/TLS** to protect sensitive email and user data.
- This communication flow ensures that data moves securely and efficiently from **email submission → analysis → threat intelligence checking → risk assessment → database storage → result and alert display**.

- Illustrate the overall workflow of the system.

The overall workflow of the **Phishing Email Detection and Cyber Threat Intelligence System** begins with collecting incoming emails and threat intelligence data from email gateways and external **STIX/TAXII feeds**. The collected data is then processed through header parsing, **NLP-based feature extraction**, and URL/attachment analysis. The extracted features are passed to the **Machine Learning and Detection Model**, which performs phishing classification, anomaly detection, and risk scoring. The detected threats are then enriched and correlated with information stored in the **Cyber Threat Intelligence Database** and SIEM system. Based on the calculated risk level, the system generates alerts and performs appropriate actions such as **automated quarantine** of malicious emails. Finally, security analysts can monitor the results through a dashboard, investigate threats, and take further security actions.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- **Scalability**

- The proposed **Hybrid Layered Client-Server Architecture** can support an increasing number of users and emails.
- Individual modules can be scaled independently when the workload increases.
- Cloud-based resources can be added when more processing power or storage is required.
- The system can handle large volumes of phishing emails without major changes to the architecture.

- **Security**

- **HTTPS/TLS** can be used to protect communication between users, servers, APIs, and external systems.
- Authentication and authorization prevent unauthorized users from accessing the system.
- Sensitive email data can be encrypted during transmission and storage.
- Access control can restrict users from viewing confidential information.
- Threat intelligence APIs help identify malicious URLs, domains, IP addresses, and other threats.

- **Performance**

- The architecture separates different processing tasks into independent modules.
- Machine learning models can quickly analyze email features and provide predictions.
- Caching can be used to reduce repeated requests to threat intelligence APIs.
- Efficient database queries can improve the speed of retrieving detection history and reports.
- The system should provide phishing detection results with minimum delay.

- **Reliability**

- The modular architecture ensures that failure in one component does not necessarily affect the entire system.
- Error handling can manage invalid emails, API failures, and database errors.
- Regular database backups can prevent loss of important information.
- System logs help identify and troubleshoot failures.
- Reliable ML and threat intelligence services improve the accuracy and consistency of detection.

- **Maintainability**

- The layered architecture makes the system easier to understand, modify, and maintain.
- Individual modules can be updated without completely redesigning the system.
- The machine learning model can be retrained or replaced when new phishing patterns appear.
- New threat intelligence APIs can be integrated without affecting other major components.
- Proper documentation, modular code, and logging make debugging and future improvements easier.

- **Availability**

- The system can be designed to remain available for users whenever phishing analysis is required.
- Cloud deployment can provide continuous access to the application.
- Backup and recovery mechanisms can help restore the system after failures.
- Redundant services can reduce downtime.
- Monitoring tools can detect system failures and performance problems quickly.

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.

Rationale Behind the Architectural Choices

- **Hybrid Layered Client-Server Architecture:** Selected because the system requires both clear separation of responsibilities and centralized processing of phishing detection and threat intelligence.
- **Layered Architecture:** Separates the system into presentation, application, detection, threat intelligence, and database layers. This makes the system easier to understand, develop, test, and maintain.
- **Client-Server Model:** Allows users to access the system through a web browser while the server performs email analysis, machine learning prediction, and threat assessment.
- **Machine Learning Component:** Used to automatically identify phishing patterns and classify emails as **Phishing, Suspicious, or Legitimate**.
- **Cyber Threat Intelligence Component:** Included to obtain updated information about malicious URLs, domains, IP addresses, hashes, and other cyber threats.
- **Database:** Used to securely store user details, email analysis results, threat intelligence information, detection history, alerts, and system logs.
- **REST APIs:** Selected to enable communication between the frontend, backend, machine learning module, database, and external threat intelligence services.
- **HTTPS/TLS Communication:** Used to protect sensitive email information and user data while it is transferred between components.
- **Modular Components:** Each component has a specific responsibility, allowing individual modules to be updated or replaced without affecting the entire system.
- **Scalability:** The architecture allows additional computing resources and services to be added as the number of users and emails increases.
- **Security:** Authentication, authorization, encryption, and secure API communication help protect the system from unauthorized access and cyber attacks.
- **Maintainability:** Separation of components makes debugging, testing, updating, and adding new features easier.
- **Reliability and Availability:** Error handling, logging, backups, monitoring, and redundant services can help keep the system operational and reduce downtime.

Conclusion: The selected architecture is appropriate because it supports the major requirements of the system, including **security, scalability, performance, reliability, maintainability, and availability**, while also allowing machine learning and cyber threat intelligence services to work together effectively.

- Discuss the trade-offs considered while selecting the architecture.

- **Performance vs. Security:** Strong security measures such as encryption, authentication, and API verification improve protection but may add a small amount of processing time.
- **Scalability vs. Cost:** Designing the system to support a large number of users and emails may require additional cloud resources, increasing the operational cost.
- **Maintainability vs. Complexity:** Separating the system into multiple layers makes maintenance easier, but it also increases the number of components and overall system complexity.
- **Centralized vs. Distributed Processing:** Centralized server processing makes management easier, while distributed processing can provide better scalability but requires more complex coordination.
- **Accuracy vs. Processing Time:** More detailed email analysis and multiple threat intelligence checks can improve detection accuracy but may increase response time.
- **Third-Party APIs vs. Dependency:** External Cyber Threat Intelligence APIs provide updated threat information, but relying on them introduces dependency on their availability, response time, and service limits.
- **Flexibility vs. Development Effort:** A modular architecture makes it easier to add new ML models and threat intelligence sources, but designing and integrating multiple modules requires additional development effort.
- **Availability vs. Cost:** Redundant servers, backups, and monitoring improve system availability but can increase infrastructure and maintenance costs.
- **Usability vs. Security:** Adding multiple authentication and security steps improves protection but may make the system slightly less convenient for users.

- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

- **Modular Architecture:** Each module performs a specific function, so new features can be added without changing the entire system.
- **Easy ML Model Updates:** The machine learning detection module can be upgraded or replaced with newer models as phishing techniques evolve.
- **New Threat Intelligence Sources:** Additional threat intelligence APIs and databases can be integrated to improve cyber threat detection.

- **API-Based Integration:** REST APIs allow the system to connect easily with external email services, security tools, threat intelligence platforms, and cloud services.
- **Database Expansion:** The database can be extended to store new types of threat indicators, detection results, user information, and security logs.
- **Scalability:** Additional servers or cloud resources can be added when the number of users and emails increases.
- **Easy Maintenance:** The layered structure allows developers to modify or troubleshoot one layer without significantly affecting other layers.
- **Technology Flexibility:** Individual components can be upgraded to newer technologies without redesigning the complete architecture.
- **Continuous Security Updates:** New security rules, phishing patterns, and threat detection techniques can be added regularly.
- **Easy Testing:** Separate modules can be tested independently, making debugging and maintenance easier.
- **Future Feature Expansion:** Features such as real-time monitoring, automated incident response, mobile applications, advanced analytics, and improved dashboards can be added later.
- **Long-Term Maintainability:** Proper modularization, documentation, logging, APIs, and separation of responsibilities ensure that the system remains easy to manage and improve over time.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

	organized, professional report.			

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25